

Advanced Programming

Assignment Submission | Spring 2026

Name	Gaurav Nath
Roll No.	CSB24073
Subject	Advanced Programming
Assignment	Assignment No. 12
Semester	Spring 2026
Language	Python 3 (Single File)

E-Commerce Order Processing System

Implements all five SOLID design principles in a single Python file with an interactive terminal interface.

Question

Design a system in Java/Python for processing customer orders in an e-commerce platform.

The order system should support:

- Multiple payment methods: Credit Card, UPI, Wallet
- Multiple notification channels: Email, SMS, Push
- Different order types: Regular, Discounted, Priority
- Ability to store order data using different storage mechanisms: Database, File

Design Constraints — Must Apply SOLID Principles:

- SRP: Each class should have a single responsibility
- OCP: Add new payment methods or notification types without modifying existing classes
- LSP: All subclasses must work correctly when used through their base type
- ISP: Design small, role-specific interfaces — avoid large interfaces
- DIP: High-level classes must depend on abstractions, not concrete implementations

The system should: create an order, process payment, send notification, and save order details.

SOLID Principles — Where Each is Applied

Principle	Class / Section	Where in Code	How it is Applied
SRP	Order	<code>class Order</code>	Only holds data — no payment, no notification logic
SRP	OrderService	<code>class OrderService</code>	Only manages the 3-step order flow
SRP	CreditCardPayment	<code>process_payment()</code>	Only handles credit card charging
SRP	EmailNotifier	<code>send_notification()</code>	Only sends email — nothing else
OCP	PaymentProcessor	<code>class PaymentProcessor(ABC)</code>	Add Bitcoin? Just subclass — touch nothing existing
OCP	Notifier	<code>class Notifier(ABC)</code>	Add WhatsApp? Just subclass — touch nothing existing
OCP	OrderStorage	<code>class OrderStorage(ABC)</code>	Add CloudStorage? Just subclass
LSP	All 3 payments	<code>process_payment(order)</code>	All work correctly as <code>PaymentProcessor</code> reference
LSP	All 3 notifiers	<code>send_notification(order)</code>	All work correctly as <code>Notifier</code> reference
ISP	PaymentProcessor	1 method only	<code>process_payment()</code> — nothing else forced
ISP	Notifier	1 method only	<code>send_notification()</code> — nothing else forced
ISP	OrderStorage	1 method only	<code>save_order()</code> — nothing else forced
DIP	OrderService	<code>__init__</code> params	Receives <code>PaymentProcessor</code> , <code>Notifier</code> , <code>Storage</code> — never instantiates them
DIP	<code>main()</code>	<code>OrderService(payment, notifier, storage)</code>	Caller injects concrete classes — <code>OrderService</code> stays abstract

Complete Source Code — main.py

Save this as main.py anywhere and run with: `python main.py`

```
# =====
# E-Commerce Order System -- SOLID Principles in Python
# Single file version
# =====

from abc import ABC, abstractmethod

# =====
# ORDER CLASS (SRP -- only holds order data)
# =====

class Order:
    def __init__(self, order_id, customer_name, amount, order_type):
        self.order_id = order_id
        self.customer_name = customer_name
        self.amount = amount
        self.order_type = order_type # Regular / Discounted / Priority
        self.status = 'PENDING'

    def __str__(self):
        return (f'[Order ID: {self.order_id} | Customer: {self.customer_name} | '
                f'Type: {self.order_type} | Amount: Rs.{self.amount} | Status: '
                f'{self.status}]')

# =====
# PAYMENT (ISP -- one abstract method | OCP -- add by subclassing)
# =====

class PaymentProcessor(ABC):
    @abstractmethod # LSP: every subclass honours this
    contract
    def process_payment(self, order) -> bool:
        pass

class CreditCardPayment(PaymentProcessor):
    def __init__(self, card_number):
        self.card_number = card_number

    def process_payment(self, order) -> bool:
        last4 = self.card_number[-4:]
        print(f' [Credit Card] Charging Rs.{order.amount} to card ending in {last4}')
        return True

class UPIPayment(PaymentProcessor):
    def __init__(self, upi_id):
        self.upi_id = upi_id

    def process_payment(self, order) -> bool:
        print(f' [UPI] Sending Rs.{order.amount} via UPI ID: {self.upi_id}')
        return True

class WalletPayment(PaymentProcessor):
    def __init__(self, wallet_id, balance):
        self.wallet_id = wallet_id
        self.balance = balance
```

```

def process_payment(self, order) -> bool:
    if self.balance < order.amount:
        print(f' [Wallet] Insufficient! Available: Rs.{self.balance}, '
              f'Required: Rs.{order.amount}')
        return False
    self.balance -= order.amount
    print(f' [Wallet] Paid Rs.{order.amount} from {self.wallet_id}. '
          f'Remaining: Rs.{self.balance}')
    return True

# =====
# NOTIFIER (ISP -- one abstract method | OCP -- add by subclassing)
# =====

class Notifier(ABC):
    @abstractmethod
    def send_notification(self, order) -> None:
        pass

class EmailNotifier(Notifier):
    def __init__(self, email):
        self.email = email

    def send_notification(self, order) -> None:
        print(f' [Email] Sent to {self.email} -- Order {order.order_id} confirmed.
,
              f'Amount: Rs.{order.amount}')
```

```

class SMSNotifier(Notifier):
    def __init__(self, phone):
        self.phone = phone

    def send_notification(self, order) -> None:
        print(f' [SMS] Sent to {self.phone} -- Order {order.order_id} confirmed.')
```

```

class PushNotifier(Notifier):
    def __init__(self, device_token):
        self.device_token = device_token

    def send_notification(self, order) -> None:
        print(f' [Push] Notification to device [{self.device_token}] -- '
              f'Order {order.order_id} confirmed.')
```

```

# =====
# STORAGE (ISP -- one abstract method | OCP -- add by subclassing)
# =====

class OrderStorage(ABC):
    @abstractmethod
    def save_order(self, order) -> None:
        pass

class DatabaseStorage(OrderStorage):
    def save_order(self, order) -> None:
        print(f' [Database] Saved: {order}')
```

```

class FileStorage(OrderStorage):
    def __init__(self, filename):
        self.filename = filename
```

```

def save_order(self, order) -> None:
    with open(self.filename, 'a') as f:
        f.write(str(order) + '\n')
    print(f'    [File] Saved to {self.filename}: {order}')

# =====
# ORDER SERVICE    (SRP + DIP -- depends on abstractions only)
# =====

class OrderService:
    # DIP: receives abstractions via constructor -- never instantiates them directly
    def __init__(self, payment_processor, notifier, order_storage):
        self.payment_processor = payment_processor
        self.notifier          = notifier
        self.order_storage     = order_storage

    def process_order(self, order):
        print(f"\n{'='*52}")
        print(f'    Processing Order: {order.order_id} | {order.order_type}')
        print(f"{'='*52}")

        print('\n >> Step 1: Payment')
        success = self.payment_processor.process_payment(order)

        if not success:
            order.status = 'PAYMENT_FAILED'
            print(f'\n >> Order {order.order_id} FAILED at payment.')
            print(' >> Saving failed order...')
            self.order_storage.save_order(order)
            return

        order.status = 'CONFIRMED'

        print('\n >> Step 2: Notification')
        self.notifier.send_notification(order)

        print('\n >> Step 3: Saving order')
        self.order_storage.save_order(order)

        print(f'\n >> Order {order.order_id} processed SUCCESSFULLY.')

# =====
# HELPER -- numbered menu
# =====

def get_choice(prompt, options):
    print(f'\n{prompt}')
    for i, option in enumerate(options, 1):
        print(f'    {i}. {option}')
    while True:
        try:
            choice = int(input('    Enter choice number: '))
            if 1 <= choice <= len(options):
                return choice
            print(f'    Please enter a number between 1 and {len(options)}.')
        except ValueError:
            print('    Invalid input. Enter a number.')

```

```

# =====
# MAIN -- interactive menu
# =====

def main():
    print('\n' + '='*52)
    print('      WELCOME TO THE E-COMMERCE ORDER SYSTEM')
    print('='*52)

    order_counter = 1

    while True:
        print(f'\n--- New Order (ID: ORD{order_counter:03d}) ---')

        # Order details
        name = input('\n Customer name: ').strip()
        while True:
            try:
                amount = float(input(' Order amount: Rs. '))
                break
            except ValueError:
                print(' Enter a valid number.')

        type_choice = get_choice('Select Order Type:', ['Regular', 'Discounted',
'Priority'])
        order_type = ['Regular', 'Discounted', 'Priority'][type_choice - 1]
        order = Order(f'ORD{order_counter:03d}', name, amount, order_type)

        # Payment
        pay_choice = get_choice('Select Payment Method:', ['Credit Card', 'UPI',
'Wallet'])
        if pay_choice == 1:
            card = input(' Enter card number: ').strip()
            payment = CreditCardPayment(card)
        elif pay_choice == 2:
            upi = input(' Enter UPI ID: ').strip()
            payment = UPIPayment(upi)
        else:
            wallet_id = input(' Enter Wallet ID: ').strip()
            while True:
                try:
                    bal = float(input(' Enter wallet balance: Rs. '))
                    break
                except ValueError:
                    print(' Enter a valid number.')
            payment = WalletPayment(wallet_id, bal)

        # Notification
        notif_choice = get_choice('Select Notification Channel:', ['Email', 'SMS',
'Push'])
        if notif_choice == 1:
            email = input(' Enter email address: ').strip()
            notifier = EmailNotifier(email)
        elif notif_choice == 2:
            phone = input(' Enter phone number: ').strip()
            notifier = SMSNotifier(phone)
        else:
            token = input(' Enter device token: ').strip()
            notifier = PushNotifier(token)

        # Storage
        store_choice = get_choice('Select Storage Mechanism:', ['Database', 'File'])
        if store_choice == 1:
            storage = DatabaseStorage()

```

```

else:
    filename = input('  Enter filename (e.g. orders.txt): ').strip()
    storage = FileStorage(filename)

    # Process -- DIP: inject all abstractions into OrderService
    service = OrderService(payment, notifier, storage)
    service.process_order(order)

    order_counter += 1

again = get_choice('Place another order?', ['Yes', 'No -- Exit'])
if again == 2:
    print('\n  Thank you for using the system. Goodbye!\n')
    break

if __name__ == '__main__':
    main()

```

How to Run

```

# Step 1: Save the code as main.py
# Step 2: Open terminal and navigate to the folder
cd 'D:\Downloads D Folder\Advanced Programming 2'

# Step 3: Run
python main.py

```

Output Screenshots

Terminal — full interactive menu (order type, payment, notification, storage selection):

```

PS D:\Downloads D Folder\Advanced Programming 2> C:\Users\gaura\AppData\Local\Python\pythoncore-3.14-64\python.exe "d:/Downloads D Folder/Advanced Programming 2/"
ECommerceSystem\Py\EComSystem.py

=====
WELCOME TO THE E-COMMERCE ORDER SYSTEM
=====

--- New Order (ID: ORD001) ---

Customer name: Gaurav Nath
Order amount: Rs.12999

Select Order Type:
1. Regular
2. Discounted
3. Priority
Enter choice number: 2

Select Payment Method:
1. Credit Card
2. UPI
3. Wallet
Enter choice number: 2
Enter UPI ID: GAURAV@OKBANK

Select Notification Channel:
1. Email
2. SMS
3. Push
Enter choice number: 2
Enter phone number: 9876543231

Select Storage Mechanism:
1. Database
2. File
Enter choice number: 2
Enter filename (e.g. orders.txt): bill.txt

=====
Processing Order: ORD001 | Discounted
=====

>> Step 1: Payment
[UPI] Sending Rs.12999.0 via UPI ID: GAURAV@OKBANK

```

```
=====
Processing Order: ORD001 | Discounted
=====

>> Step 1: Payment
[UPI] Sending Rs.12999.0 via UPI ID: GAURAV@OKBANK

>> Step 2: Notification
[SMS] Sent to 9876543231 - Order ORD001 confirmed.

>> Step 3: Saving order
[File] Saved to 'bill.txt': [Order ID: ORD001 | Customer: Gaurav Nath | Type: Discounted | Amount: Rs.12999.0 | Status: CONFIRMED]

>> Order ORD001 processed SUCCESSFULLY.

Place another order?
1. Yes
2. No - Exit
Enter choice number: 

Ln 2, Col 51  Spaces: 4  UTF-
```

Terminal — wallet payment failure (insufficient balance):

```
=====
Processing Order: ORD002 | Regular
=====

>> Step 1: Payment
[wallet] Insufficient balance! Available: Rs.200.0, Required: Rs.5000.0

>> Order ORD002 FAILED at payment.
>> Saving failed order...
[Database] Saved: [Order ID: ORD002 | Customer: Test user | Type: Regular | Amount: Rs.5000.0 | Status: PAYMENT_FAILED]

Place another order?
1. Yes
2. No - Exit
```